

Modul 06: CQRS-Implementierung für Bankkonto-API

Systemvoraussetzungen: Visual Studio, .NET

Ziel: Entwicklung einer Web-API mit getrennten Lese- und Schreibpfaden unter Verwendung von MediatR.

Hinweise: Bei diesen Labs geht es nicht um die vollständige Implementierung, sondern ein Verständnis der Konzepte zu entwickeln. Bitte begründen Sie Ihre Entscheidungen und gewählte Designoptionen. Die Entwürfe können in der Feedbackrunde diskutiert werden.

1 Projekt anlegen und Dependencies registrieren

Legen Sie ein neues Web-API Projekt an und fügen Sie entweder das vorherige Lab oder die Musterlösung als Projektreferenz hinzu. Installieren Sie zudem das NuGet-Paket MediatR und konfigurieren Sie es so:

```
builder.Services.AddMediatR(config =>
    config.RegisterServicesFromAssembly(typeof(Program).Assembly));
```

Registrieren Sie den HttpContextAccessor, EventStore und eine BankAccountFactory für die Dependency Injection in der Program.cs. Die Factory soll den BankAccount erstellen und die dafür notwendige AccountId aus dem HttpRequest auslesen. Die Methode um einen Account aus einer existierenden Id zu erzeugen könnte so aussehen:

```
public BankAccount CreateFromRouteParam(string key = "accountId")
{
    var accountIdStr = _httpContext.Request.RouteValues[key]?.ToString()
        ?? _httpContext.Request.Query[key].ToString();

    if (Guid.TryParse(accountIdStr, out var accountId))
        return new BankAccount(_eventStore, accountId);

    throw new KeyNotFoundException($"{key} missing in request data.");
}
```

2 Commands und Handlers hinzufügen

Implementieren Sie die Commands ([IRequest](#)) und die jeweiligen Handler ([IRequestHandler<, >](#)) dazu.

- OpenAccountCommand : IRequest<Guid>
- DepositCommand : IRequest
- WithdrawCommand : IRequest
- CloseAccountCommand : IRequest

3 Queries und Handlers hinzufügen

Implementieren Sie die Queries ([IRequest](#)) und die jeweiligen Handler ([IRequestHandler<, >](#)) dazu.

- GetBalanceQuery : IRequest<decimal>
- GetStatementQuery : IRequest<IEnumerable<BankAccountEventDto>>

Für das Statement soll ein sogenanntes DTO (Data Transfer Object) verwendet werden, um die Daten gezielt dem Client zu offenbaren.

4 Account Controller hinzufügen

Der AccountController bildet die Schnittstelle zwischen Geschäftslogik mittels MediatR und Web-API.

```
[ApiController]
[Route("api/accounts")]
public class BankAccountController : ControllerBase
{
    private readonly IMediator _mediator;

    [HttpPost]
    public async Task<IActionResult> OpenAccount(
        [FromBody] OpenAccountCommand command)
    {
        var accountId = await _mediator.Send(command);
        return CreatedAtAction("GetBalance", new { accountId }, accountId);
    }

    [HttpGet("{accountId:guid}/balance")]
    public async Task<decimal> GetBalance(Guid accountId)
        => await _mediator.Send(new GetBalanceQuery(accountId));

    // TODO: Implementieren Sie Endpoints für Einzahlung/Auszahlung
}
```

5 Web-API testen

Sie können die Web-API mit Swagger, Postman, Insomnia, Hoppscotch testen. Eine weitere Möglichkeit sind [http-files](#), welche für dieses Lab wie folgt aussehen kann. (Bitte Port noch anpassen)

```
@WebApi_HostAddress = http://localhost:5555
@AccountId = BD300B3B-9640-45C5-8BFF-F803FE98419F

### Open Account
POST {{WebApi_HostAddress}}/api/accounts
Content-Type: application/json

{
  "initialBalance": 1000
}

### Deposit Money
POST {{WebApi_HostAddress}}/api/accounts/{{AccountId}}/deposit
Content-Type: application/json

{
  "accountId": "{{AccountId}}",
  "amount": 200
}

### Withdraw Money
POST {{WebApi_HostAddress}}/api/accounts/{{AccountId}}/withdraw
Content-Type: application/json

{
  "accountId": "{{AccountId}}",
  "amount": 500
}

### Get Balance
GET {{WebApi_HostAddress}}/api/accounts/{{AccountId}}/balance

### Get Account History
GET {{WebApi_HostAddress}}/api/accounts/{{AccountId}}/statement

### Close Account
POST {{WebApi_HostAddress}}/api/accounts/{{AccountId}}/close
```